

OE-EC704A — Web Technology

Semester VII • 3L:0T:0P • 3 Credits

Module 2 Exam Notes: Introduction to Java Programming & OOP

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Java Execution Environment	2
1.1	JDK vs. JRE vs. JVM	2
2	Java Syntax and Control Flow	3
2.1	Primitive vs. Reference Data Types	3
2.2	Memory Allocation Model	3
2.3	Control Flow Statements	4
3	Class Fundamentals and Variables	4
3.1	Static vs. Instance Members	4
3.2	Inner Classes	4

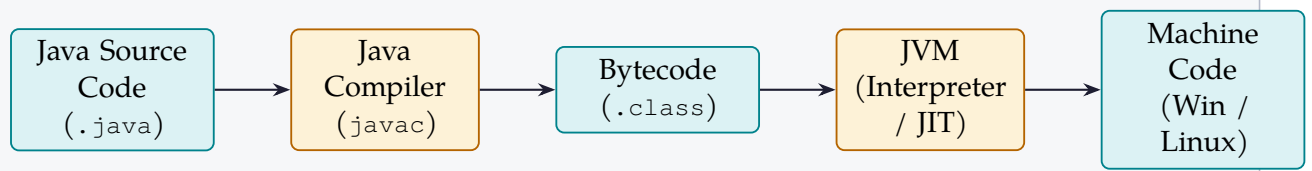
1. Java Execution Environment

Java uses a hybrid execution model combining compilation and interpretation. This ensures its core principle: **Write Once, Run Anywhere (WORA)**.

Bytecode and Platform Independence

- **Bytecode:** A highly optimized set of machine instructions designed for execution by the JVM. It is stored in a `.class` file after compilation.
- **Platform Independence:** Java source code compiles into platform-independent bytecode. However, the **JVM itself is platform-dependent** because it must translate bytecode into the host system's native machine instructions.

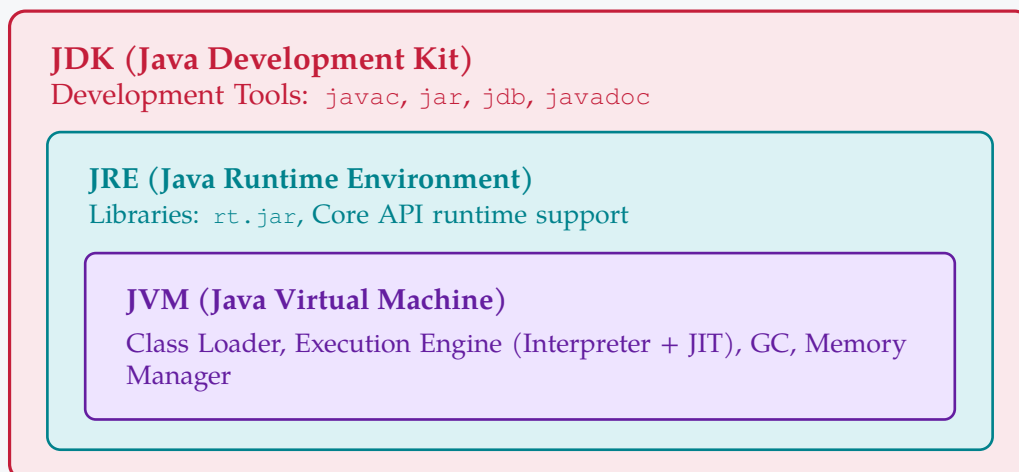
Java Compile-and-Run Execution Flow



1.1 JDK vs. JRE vs. JVM

Understanding the boundary between JVM, JRE, and JDK is crucial:

JDK, JRE, and JVM Containment Relationship



Metric	JVM (Virtual Machine)	JRE (Runtime)	JDK (Development)
Composition	Classloader, Execution Engine, Garbage Collector.	JVM + Core class libraries (rt.jar) + plugins.	JRE + Compiler (javac) + tools (jar, jdb).
Purpose	Execution of compiled bytecode.	Running Java applications.	Developing and running Java apps.
Target User	None (internal engine).	End users/clients.	Developers.

2. Java Syntax and Control Flow

2.1 Primitive vs. Reference Data Types

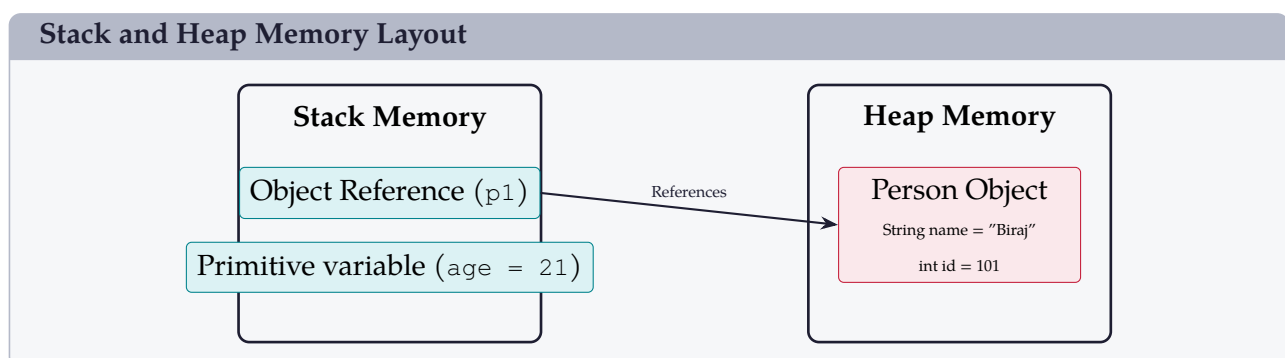
Java enforces static type checking and groups types into two primary categories:

Feature	Primitive Data Types	Reference Data Types
Allocation	Stored directly on the stack memory.	Object on Heap; address reference on Stack.
Default Value	0, 0.0, or false (never null).	null by default.
Size	Fixed sizing (e.g., int: 32-bit, double: 64-bit).	Depends on the host system architecture.
Examples	byte, short, int, long, float, double, char, boolean	Classes, interfaces, arrays, strings.

2.2 Memory Allocation Model

Java allocates memory across two distinct spaces during runtime:

- **Stack Memory:** Stores primitive variables and local references. Uses LIFO allocation.
- **Heap Memory:** Dynamic memory pool that hosts all runtime objects and instance variables.



2.3 Control Flow Statements

Java supports selection (`if-else`, `switch`), iteration (`for`, `while`, `do-while`), and jump (`break`, `continue`, `return`) statements:

- **break:** Exits the innermost loop or switch statement immediately.
- **continue:** Skips the current iteration and jumps directly to the next condition evaluation.
- **Labeled Statements:** Loops can be labeled to specify which outer loop a `break` or `continue` targets (e.g., `outerLoop: for(...)`).

3. Class Fundamentals and Variables

Class, Object, and Constructor

- **Class:** A blueprint or template that defines the structure and behaviors (fields and methods) of objects.
- **Object:** A physical instance of a class, holding state values and responding to message calls.
- **Constructor:** A special block of code resembling a method, invoked during object instantiation via the `new` keyword. It must match the class name exactly and has no return type.

3.1 Static vs. Instance Members

Members marked with the `static` keyword behave differently from normal instance variables:

Feature	Static Members	Instance Members
Memory Allocation	Allocated once in the Metaspace/Method area.	Allocated inside each object on the Heap.
Access Syntax	Accessed via class name (e.g., <code>Math.sqrt()</code>).	Requires an object instance to access.
Lifecycle	Lives from class load to class unload.	Tied to object lifetime; garbage collected.
Context	Cannot reference <code>this</code> or <code>super</code> .	Full access to static and instance members.

3.2 Inner Classes

An inner class is a class declared inside another outer class:

- **Member Inner Class:** Non-static nested class. Holds a reference to the outer class instance and can access all its private members directly.
- **Static Nested Class:** Declared static. Cannot access non-static outer members directly.

- **Anonymous Inner Class:** A class without a name defined inline to implement interfaces or extend abstract classes.

Quick Revision — Module 2

Introduction to Java Programming & OOP | OE-EC704A

Java Class & Loop Syntax

Concept	Syntax and Structural Format
Class Declaration	<code>class Person { String name; Person(String n) { name = n; } }</code>
Main Method Signature	<code>public static void main(String[] args) { ... }</code>
Object Instantiation	<code>Person p1 = new Person("Biraj");</code>
Static Method Call	<code>ClassName.staticMethodName();</code>
Loop Statements	Conditional and Iterative Formats
for Loop	<code>for(initialization; condition; increment/decrement) { ... }</code>
while Loop	<code>while(condition) { statements; }</code>
do-while Loop	<code>do { statements; } while(condition);</code>
Labeled Break	<code>outer: for(...) { if(cond) break outer; }</code>

One-Line Definitions

- **JVM (Java Virtual Machine):** An abstract machine that parses and executes compiled Java bytecode.
- **Bytecode:** Standard intermediate instruction format generated by the compiler.
- **JDK:** Comprehensive toolset containing JRE and compiler scripts for developing Java apps.
- **JRE:** Run runtime libraries and JVM package required to execute Java bytecode.
- **Class:** User-defined blueprint declaring structural fields and behavior routines.
- **Object:** An active, stateful runtime instance of a class allocated on the heap.
- **Constructor:** Instantiation routine that initializes object state variables during creation.
- **Garbage Collector:** Automatic JVM system thread that frees heap memory by sweeping unreferenced objects.

Frequently Asked / Viva Questions

Q1. Why is Java platform-independent but JVM is platform-dependent?

Java code compiles to bytecode which runs on any JVM, regardless of the host. However, the JVM must be tailored to the specific OS architecture to translate that bytecode to native machine instructions.

Frequently Asked / Viva Questions (contd.)**Q2. What is the difference between `path` and `classpath` variables?**

`path` is an OS environment variable locating command-line executables like `javac` and `java`. `classpath` instructs the JVM where to look for compiled user classes and libraries.

Q3. Can a class have multiple constructors? If so, how are they distinguished?

Yes, this is known as constructor overloading. They are distinguished by their signatures: the number, type, and order of parameters.

Q4. What is the default constructor and when is it generated?

If no constructor is declared, the compiler automatically generates a no-argument default constructor that initializes fields to default values. If any constructor is declared, this default constructor is not created.

Q5. What is the difference between `break` and `continue` statements?

`break` exits a loop block immediately, while `continue` skips only the current iteration and evaluates the loop condition for the next cycle.

Q6. What is the purpose of the `static` keyword in Java?

It binds variables or methods to the class itself, rather than to instances. They are loaded once in class metadata space.

Q7. Can a static method access non-static instance variables? Why?

No. Static methods run without any object instance context, so they do not have access to the `this` reference necessary to locate instance variables.

Q8. What is the difference between pass-by-value and pass-by-reference in Java?

Java is strictly pass-by-value. For primitives, a copy of the value is passed. For reference types, a copy of the heap reference address is passed, not the object itself.

Q9. What is the purpose of garbage collection in Java?

It runs asynchronously to track and reclaim heap memory occupied by objects that have become unreachable from active threads.

Q10. Why does Java not support global variables?

To prevent global namespace pollution and promote modular encapsulation, Java requires every variable to reside inside a class scope.

Java Iteration Statement Comparisons

Feature	for Loop	while Loop	do-while Loop
Loop Structure	Counter-based iteration.	Condition-controlled iteration.	Exit-controlled iteration.
Condition Check	Evaluated at start.	Evaluated at start.	Evaluated at end.
Min. Runs	0 runs if condition fails.	0 runs if condition fails.	Guaranteed at least 1 run.
Best Use Case	Iterating when count is known.	Loop until event triggers.	Interactive menus.